The technology behind your NetOps Assistant

Every model, agent, method and algorithm in the project, explained from zero — what it is, how it works, and exactly how your project uses it

For	Someone new to AI, machine learning and web technology who needs to understand and defend the project
Covers	32 technologies in 7 parts: foundations · data and rules · search · machine learning · LLM and agents · quality · serving and interface
Every topic	In one sentence → everyday analogy → how it works → how your project uses it → what to say if the panel asks
Numbers	Taken from your project: its configuration files, trained models, evaluation results and real outputs. Charts marked 'real' were computed from your data for this document.

Companion to docs/Project_Guide.pdf (what was built, phase by phase). This primer explains the ideas underneath. Rebuild it with python -m docs.tech_primer.build_primer.

How to read this primer

You do not need any background. Each topic starts with a one-sentence definition and an everyday analogy, then explains the mechanics, then shows the exact place, settings and numbers in your project. If you only have ten minutes before the panel, read the one-sentence definitions and the green **If the panel asks** boxes.

Box	Meaning
Everyday analogy (purple)	An intuition from daily life. Useful for explaining, never a substitute for the real definition.
If the panel asks (green)	A short, accurate answer you can say out loud.
Watch out (orange)	A common misunderstanding, or a limitation you should state honestly.

Words in `this font` are names of files, settings or code in your project, for example `src/config/thresholds.yaml`. The glossary at the end defines every technical term.

Contents

How to read this primer . . . 2

Part A — Foundations . . . 5

1. The big picture: three kinds of intelligence . . . 5
2. Data words: record, feature, target, label . . . 7

Part B — Data and rules . . . 8

3. pandas, Parquet and data validation . . . 8
4. Percentiles and derived conditions . . . 9
5. Rule-based scoring: category, severity and priority . . . 10

Part C — Search and retrieval . . . 12

6. Turning rows into searchable text . . . 12
7. Embeddings and Sentence-Transformers . . . 13
8. Vector database: ChromaDB and HNSW . . . 14
9. Keyword search: BM25 . . . 15
10. Metric similarity and percentile rank . . . 17
11. Hybrid retrieval and query understanding . . . 18

Part D — Machine learning . . . 20

12. Supervised learning, train/test split and cross-validation . . . 20
13. Linear regression, logistic regression and feature encoding . . . 21
14. Decision trees and random forests . . . 22
15. Gradient boosting (HistGradientBoosting) . . . 23
16. Measuring models: R^2, MAE, RMSE, accuracy, precision, recall, F1 . . . 24
17. Data leakage and feature sets . . . 25
18. Prediction intervals and risk bands . . . 26

Part E — LLM, RAG and agents . . . 28

19. Large language models: gpt-oss-20b on NVIDIA . . . 28
20. Retrieval-augmented generation (RAG) . . . 29
21. The grounding gate: catching hallucinations . . . 30
22. LLM-as-a-judge . . . 31
23. Root-cause hypotheses: the playbook heuristic . . . 32
24. Agents and orchestration . . . 33
25. Guardrails and prompt injection . . . 34
26. Provenance: where every statement comes from . . . 35

Part F — Quality: testing and evaluation . . . 36

27. Automated testing with pytest . . . 36
28. Evaluating retrieval and answers . . . 37

29. DeepEval: LLM-judged quality metrics 38

Part G — Serving and interface 39

30. APIs: REST, FastAPI and Pydantic 39

31. The web interface: React, TypeScript and Vite (and Streamlit) 40

32. Supporting tools: YAML, .env, SQLite and Docker 41

Cheat sheet — one sentence per technology 43

Glossary 44

Part A — Foundations

The vocabulary every later chapter uses, and the big picture of how the pieces fit together.

1. The big picture: three kinds of intelligence

In one sentence. Your project combines hand-written **rules**, trained **machine-learning models** and a **large language model (LLM)**, and gives each one only the job it is best at.

> **Everyday analogy**
> A hospital: written clinical guidelines (rules) decide what counts as a fever; lab equipment calibrated on past patients (ML) estimates risk; a doctor (the LLM) explains the results to you in plain words — but the doctor is not allowed to invent lab values.

How it works

Rules are instructions a person writes: "latency above 129.92 ms is high latency". They are exact, explainable and easy to test, but someone must decide every threshold.

Machine learning (ML) lets a computer find patterns in examples. You show it 80,000 past records together with their fault rate, and it learns a function that estimates the fault rate of a new record.

A large language model is a very large neural network trained on huge amounts of text. It is excellent at reading and writing language, and also good at sounding confident about things that are not true. That is why your project never lets it decide facts.

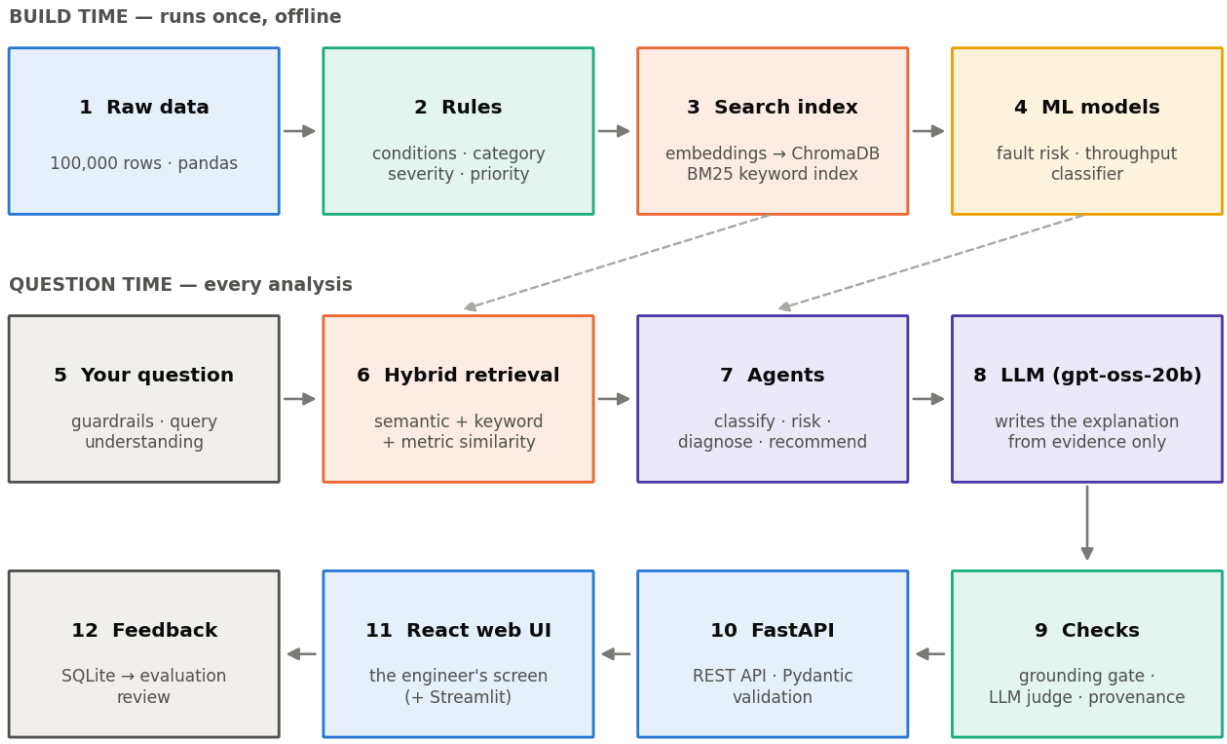


Figure 1.1 — How the pieces connect. Steps 1–4 run once when the data is built; steps 5–12 run for every question (illustration).

How your project uses it

- **Rules** decide the facts about each record: the 18 conditions, the category, severity and priority (Part B), and the root-cause hypotheses (Chapter 23).
- **ML** makes estimates the rules cannot: fault risk, throughput and a cross-check of the category (Part D).
- **The LLM (NVIDIA gpt-oss-20b)** only writes the explanation and reviews it (Part E). Code checks every number it writes.
- **Search** (Part C) connects them: it finds the historical records that serve as evidence.

> **If the panel asks**
> "Facts come from the data and documented rules, estimates come from ML models with measured error, and the LLM only explains — a deterministic check rejects anything it writes that is not in the evidence."

2. Data words: record, feature, target, label

In one sentence. Machine learning and search both work on **tables**: each row is one example (a **record**), each column is one measured property (a **feature**), and the column you want to predict is the **target**.

> **Everyday analogy**
> A spreadsheet of house sales: each row is a house; size, rooms and location are features; the sale price is the target a model learns to predict.

How it works

Raw data is what was measured. **Derived data** is computed from it by your own rules. Keeping them apart is the most important habit in your project: raw values are facts, derived values are interpretations.

A **label** is the correct answer attached to a training example. In many projects people write labels by hand. Your dataset has **no** human labels, so every category and severity is derived by rules — and the project says so everywhere.

Training means fitting a model on examples; **inference** (or prediction) means using the trained model on a new case.

How your project uses it

Dataset: `fenar/5g_netops`, 100,000 records × 27 raw columns, downloaded at a pinned version and checked with a SHA-256 fingerprint so it can never change silently.

- Raw features: KPIs such as `latency_ms`, `packet_loss_pct`, `handover_success_pct`; context such as season, weather, temperature, state and city.
- Targets: `fault_rate_pct` (fault-risk model), `throughput_mbps` (throughput model) and the derived `primary_category` (classifier).
- Each record gets a stable ID, `ROW-000001` to `ROW-100000`, so every statement in an answer can point to a real row.

> **Watch out**
> The data is synthetic: columns are uniformly distributed, locations are random, and the fault rate is almost a linear formula of seven other columns. Say this yourself before anyone else does — it is why causes stay hypotheses.

> **If the panel asks**
> "There are no human labels, so I derived them with documented rules and I always call them derived. Raw values are never modified, and each record keeps an ID back to its row in the original file."

Part B — Data and rules

How raw measurements become clean, typed data and then the conditions, categories, severity and priority that everything else relies on.

3. pandas, Parquet and data validation

In one sentence. pandas is the standard Python library for working with tables; **Parquet** is a compact file format for storing them; **validation** means checking every value against written rules before using it.

> **Everyday analogy**
> Receiving goods at a warehouse: you check every box against the delivery note (validation), then store it on labelled shelves (Parquet) so it is quick to find later.

How it works

pandas loads a table into a **DataFrame** — rows and named columns — and lets you filter, compute and summarise with a few lines of code.

Parquet stores each column together and remembers its type (number, text, true/false), so tables load quickly and a number never comes back as text.

A **schema** is the written contract for the data: column names, types, units and valid ranges.

How your project uses it

The schema lives in `src/config/schema.yaml`: for example signal strength must lie between −140 and −44 dBm and percentages between 0 and 100.

- `src/preprocessing/validator.py` checks types, ranges, missing values and unknown categories. On your data it found 0 errors in 100,000 rows; nothing was dropped and nothing was filled in (no imputation).
- `src/preprocessing/cleaner.py` standardises names and types; four `dq_*` columns flag suspicious but valid patterns (for example more critical alarms than total alarms) instead of changing them.
- Output: `data/processed/records.parquet` — 27 typed raw fields + record_id + 4 quality flags.

> **Problem found**
> A real bug found by testing: pandas' fast number parser changed about 15% of values in their last decimal place. A test that compares every processed value with the raw CSV text caught it, and the fix parses numbers exactly with Python's own float().

> **If the panel asks**
> "Every processed value is proven identical to the raw text by a test, so preprocessing cannot fabricate or alter a measurement."

4. Percentiles and derived conditions

In one sentence. A **percentile** tells you where a value stands among all values: the 90th percentile (P90) is the value that 90% of records are below. Your project marks the worst 10% of each KPI as a named **condition** such as `high_latency`.

> **Everyday analogy**
> Growth charts for children: a child above the 90th percentile for height is tall *compared with other children of the same age*, not by an absolute rule.

How it works

Sort all 100,000 latency values; the value at position 90,000 is P90. Anything above it is in the worst decile (tenth).

For metrics where low is bad (throughput, handover success) the worst decile is below P10 instead.

Why not industry rules of thumb? Your Phase 1 analysis showed that on this dataset common operator targets would flag 80–88% of rows — a condition that is true for almost everyone carries no information.

One exception is a **domain threshold**: weak signal is fixed at −110 dBm, an established radio engineering boundary that is also selective on this data.

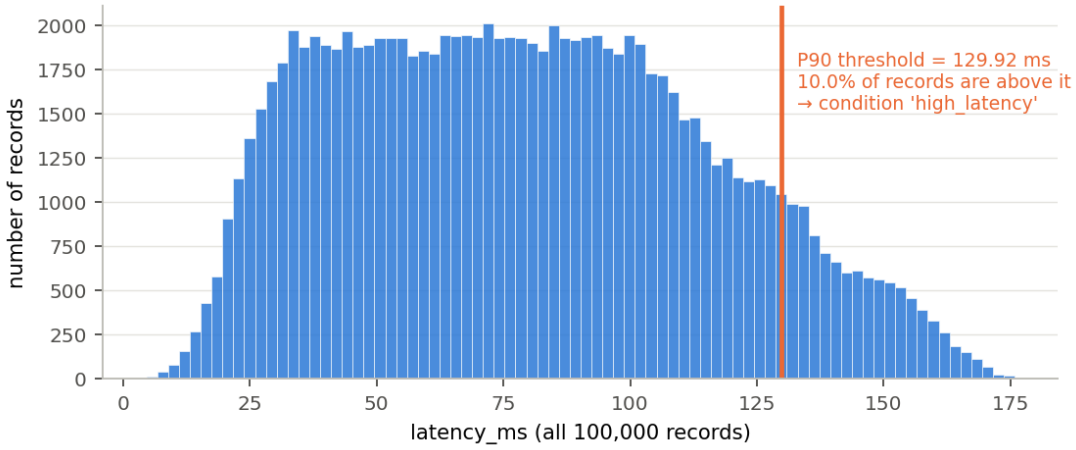


Figure 4.1 — Real: latency of all 100,000 records. The P90 threshold (129.92 ms) marks exactly the worst 10% as 'high latency'.

How your project uses it

18 conditions in `src/config/thresholds.yaml`, computed once from the data and re-checked by `python -m src.preprocessing.build_thresholds --check` so the file cannot drift from the data.

Each record gets 18 true/false columns (`cond_high_latency` …). A missing value never triggers a condition; it is reported as unavailable.

The same function is used for the whole dataset and for values typed into the web UI, so there is only one implementation of every rule.

> **If the panel asks**
> "A condition means 'in the worst 10% of this network's own history', computed from the data and documented in YAML — except weak signal, which uses the standard −110 dBm boundary."

5. Rule-based scoring: category, severity and priority

In one sentence. Three transparent point systems turn a record's conditions into a **category** (what kind of problem), a **severity** (how badly the network behaves) and a **priority** (how urgently to act).

> **Everyday analogy**
> A hospital triage nurse: symptoms decide the department (category), how abnormal the vital signs are decides severity, and severity plus how many people are affected decides who is seen first (priority).

How it works

Category — weighted scoring. Each category lists condition weights in `taxonomy.yaml`, e.g. Mobility / Handover = 1.2 × low_handover + 0.4 × high_call_drop. The highest score wins; other positive scores become secondary categories; no condition means Normal. Scoring was chosen over 'A AND B' rules because conditions rarely occur together on this data.

Severity — points × badness. Each condition has points (low availability 30, call drops 22, … parameter changes 4). Badness is 0.5 when a value just crosses its threshold and rises to 1.0 at the dataset's 1% extreme, so a slightly bad value earns half the points. Levels: LOW below 15, MEDIUM from 15, HIGH from 35, CRITICAL from 55, with overrides so one extreme problem is never diluted.

Priority — severity plus impact. Priority adds up to 15 points for affected users (scaled by the dataset's P99 of 9,902 users), 10 if a customer reported an issue, 10 for low availability, 5 for many critical alarms, and 0–15 for the predicted fault-risk level. P1 from 75, P2 from 50, P3 from 28, otherwise P4.

How your project uses it

Worked example, record `ROW-000001` (real):

Step	Calculation	Result
Conditions	weak signal (−110.12 dBm vs −110), many parameter changes (18 vs ≥ 18), high data usage	3 conditions
Category	weak_signal 1.0 beats Congestion / Load 0.8 and Configuration 0.7	Radio / Signal Quality
Severity	weak signal 12 points × badness 0.51 = 6.1; parameter changes 2.0; customer issue 5	13.1 → LOW
Priority	13.1 + users 2,442 / 9,902 × 15 = 3.7 + customer issue 10	26.8 → P4

Badness here is only 0.51 because −110.12 dBm has barely crossed the −110 threshold; at the extreme (−119.41 dBm) it would earn all 12 points.

If the panel asks

"Category, severity and priority are documented point systems in YAML, not ML and not an LLM. Severity measures how badly the network behaves; priority adds impact, so they answer different questions."

Part C — Search and retrieval

How the system finds the historical records most similar to a question. This is the 'R' in RAG and the foundation of every answer.

6. Turning rows into searchable text

In one sentence. Search engines and embedding models read text, but your dataset is numbers, so every record is described in a deterministic sentence built only from its own fields and derived features.

> **Everyday analogy**
> A library catalogue card: it does not contain the book, but a short, consistent description that lets you find the right book quickly.

How it works

Deterministic means the same record always produces exactly the same text — no randomness and no LLM, so no invented content.

Two versions are produced because two search methods want different things (next chapters).

How your project uses it

- `search_text` — the full description: conditions, category, severity, priority, location, season, weather and all metrics rounded to two decimals. Shown to users and the LLM, and indexed by BM25 keyword search.
- `embedding_text` — only the qualitative part (no numbers), for the embedding model. Real example for ROW-000001:

```
Derived conditions: weak signal strength (weak_signal), high data usage (high_data_usage), many
parameter changes (high_parameter_changes). Derived category: Radio / Signal Quality (secondary:
Congestion / Load, Configuration). Derived severity: LOW. Derived priority: P4. Weather Clouds,
season Winter. Customer issue reported: yes.
```

Numbers are left out of the embedding text because embedding models read only the first 256 word pieces and capture meaning poorly from digits. Numeric closeness is handled separately by metric similarity (Chapter 10).

Many records share the same qualitative description: only 27,117 distinct embedding texts exist for 100,000 records, so each is embedded once and reused.

> **If the panel asks**
> "Records are described by a fixed template from their own fields, so the search text can never contain anything that is not in the data."

7. Embeddings and Sentence-Transformers

In one sentence. An **embedding** turns a piece of text into a list of numbers (a **vector**) so that texts with similar meaning get similar vectors. Your project uses the model `all-MiniLM-L6-v2` from the Sentence-Transformers library, which produces 384 numbers per text.

> **Everyday analogy**
> GPS coordinates for meaning: 'long network delay' and 'high latency' are parked on the same street, while 'snowfall in winter' is in a different town.

How it works

MiniLM is a small **transformer** neural network (6 layers). It was trained on over a billion pairs of related sentences to place related sentences close together — this is called **contrastive learning**.

Closeness is measured with **cosine similarity**: 1 means pointing the same way (same meaning), 0 means unrelated. Your code normalises every vector to length 1, so cosine similarity is simply the dot product (multiply matching numbers and add them up).

It runs on the CPU, needs no internet once downloaded, and embeds a short text in milliseconds.

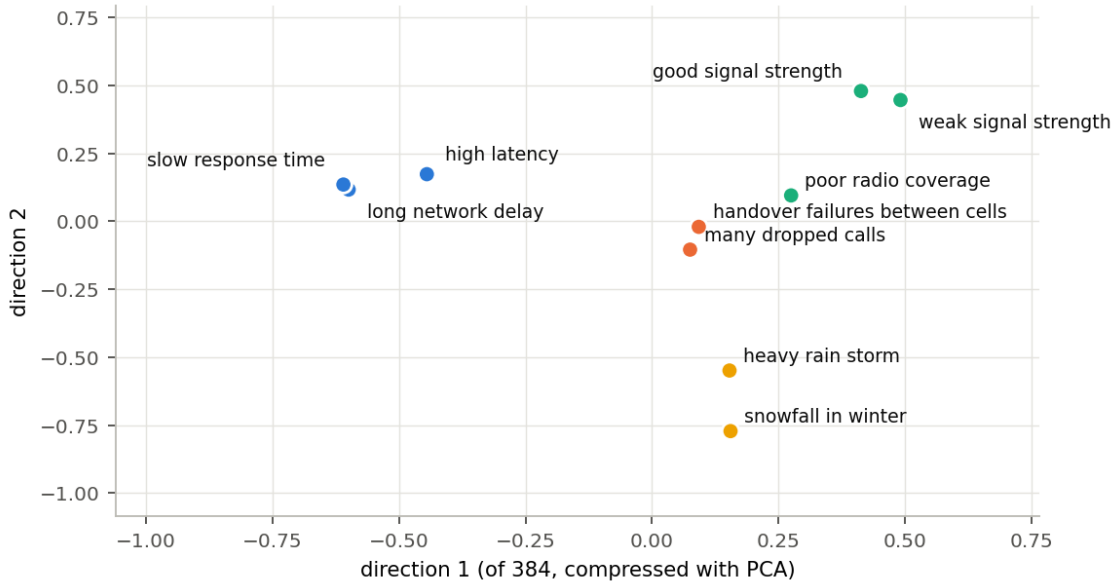


Figure 7.1 — Real: ten phrases embedded by your model, compressed from 384 to 2 directions with PCA. Phrases with the same meaning cluster.

How your project uses it

Settings in `src/config/retrieval.yaml`: model all-MiniLM-L6-v2, batch size 128. Code: `src/rag/embeddings.py`.

The real similarities show both the strength and the blind spot: 'high latency' vs 'long network delay' = 0.60 (different words, same meaning — good), but 'weak signal strength' vs 'good signal strength' = 0.77, higher than 'weak signal strength' vs 'poor radio coverage' = 0.36. Embeddings notice the topic (signal) more than the direction (good or weak).

That is exactly why your project does not rely on embeddings alone: keyword search and metric similarity cover this blind spot, and the query parser removes 'good/normal' phrases before semantic search (Chapter 11).

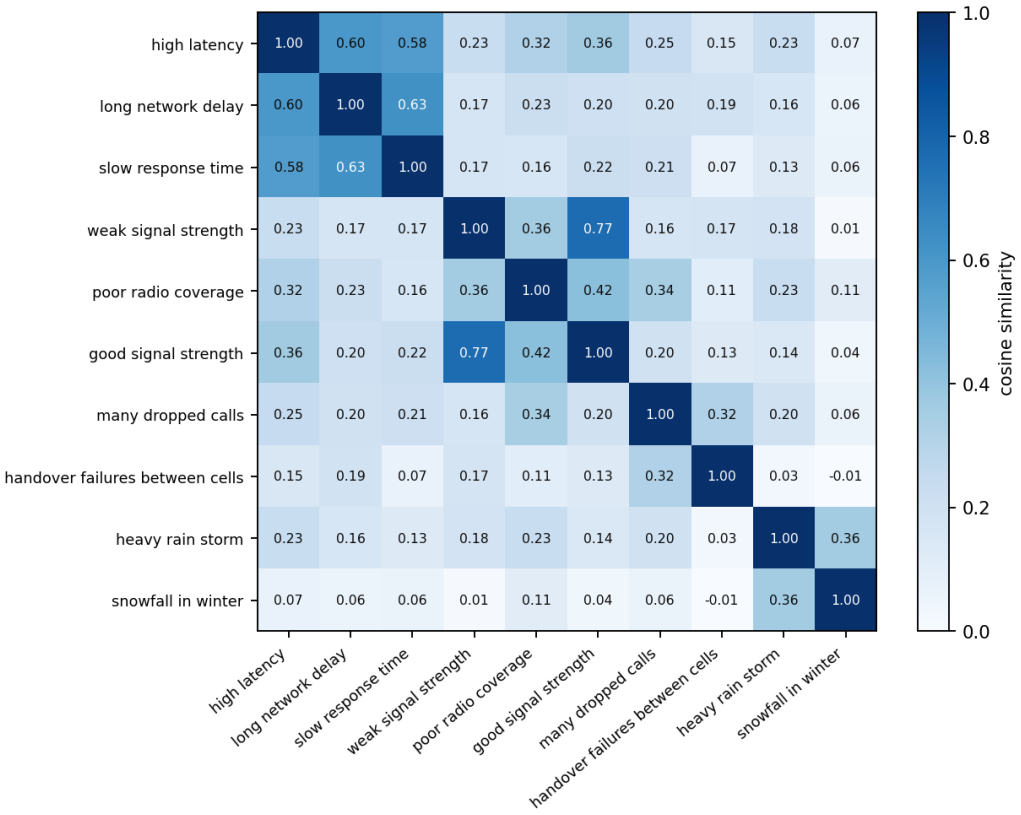


Figure 7.2 — Real: cosine similarity between every pair of phrases (1.00 = identical meaning).

> **If the panel asks**
> "Embeddings capture meaning, so 'declining handover' finds 'low handover success'. They are weak at negation and numbers, which is why they are only one of three signals in hybrid retrieval."

8. Vector database: ChromaDB and HNSW

In one sentence. A **vector database** stores millions of embeddings and quickly finds the ones closest to a query vector. Your project uses **ChromaDB**, which uses an index called **HNSW** to avoid comparing the query with every record.

> **Everyday analogy**
> Finding the nearest café in a city: you take the motorway to the right district, then main roads, then side streets — instead of checking every café in the country.

How it works

Comparing a query with 100,000 vectors one by one works but becomes slow at scale. **Approximate nearest-neighbour** search accepts a tiny chance of missing the very best match in exchange for speed.

HNSW (Hierarchical Navigable Small World) builds layers of graphs. The top layer has few points linked by long jumps; each lower layer has more points and shorter links; the bottom layer contains every vector. A search starts at the top and greedily moves closer at every layer.

Each stored item can carry **metadata** (season, weather, state …). A **where filter** limits the search to items whose metadata match, e.g. only Minnesota in winter.

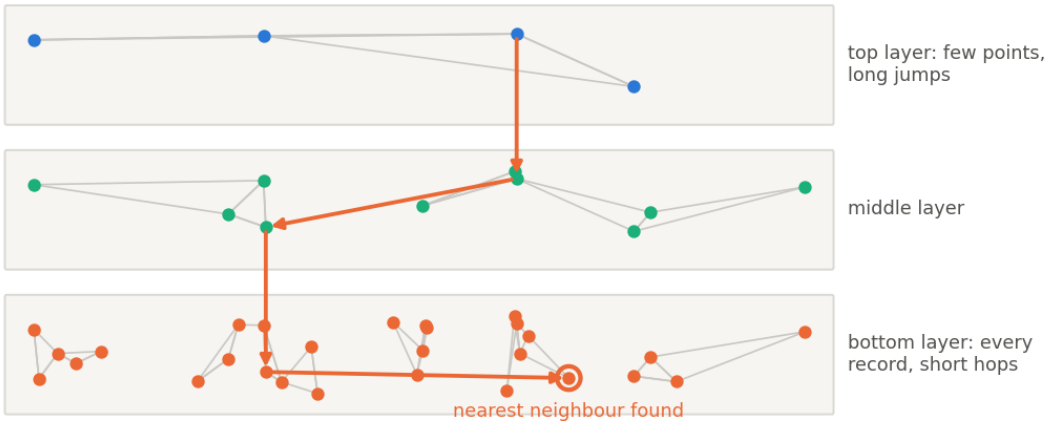


Figure 8.1 — How an HNSW search narrows down layer by layer (illustration).

How your project uses it

- `src/rag/vector_store.py`: collection `network_records`, cosine distance, stored in `data/processed/chroma` (100,000 vectors × 384 numbers).
- Each item's ID **is** the record_id, so a search hit maps directly to a real record — no separate lookup table that could drift.
- Metadata stored with every vector: season, weather, state, city, category, severity, priority, customer issue and the 18 condition flags, so filters run inside the vector search.
- The first search after start-up loads the index into memory, which is why the API 'warms up' for about 40 seconds before it accepts questions.

> **If the panel asks**
> "ChromaDB stores one 384-number vector per record with its metadata. HNSW finds close vectors without scanning all 100,000, and metadata filters restrict the search to the user's state, season or weather."

9. Keyword search: BM25

In one sentence. BM25 is the classic ranking formula behind search engines: a record scores high when it contains the query's words, especially rare words, but repeating a word many times gives diminishing returns.

> **Everyday analogy**
> Searching a library for 'monsoon flooding': a book that mentions 'flooding' ten times is relevant, but not ten times more relevant than one mentioning it five times — and the rare word 'monsoon' matters more than the common word 'the'.

How it works

Tokenisation splits text into words (tokens), lower-cases them and removes stop words such as 'the' and 'and'.

Term frequency (tf) — how often the word appears in this record. BM25 makes it **saturate** with the parameter k1: the first mentions count a lot, later ones less.

Inverse document frequency (IDF) — words that appear in few records get a high weight; a word in every record gets almost zero.

Length normalisation with parameter b — long documents are slightly penalised so they do not win just by containing more words.

Score for one word = IDF × tf × (k1 + 1) / (tf + k1 × (1 − b + b × length / average length)); the record's score is the sum over the query's words.

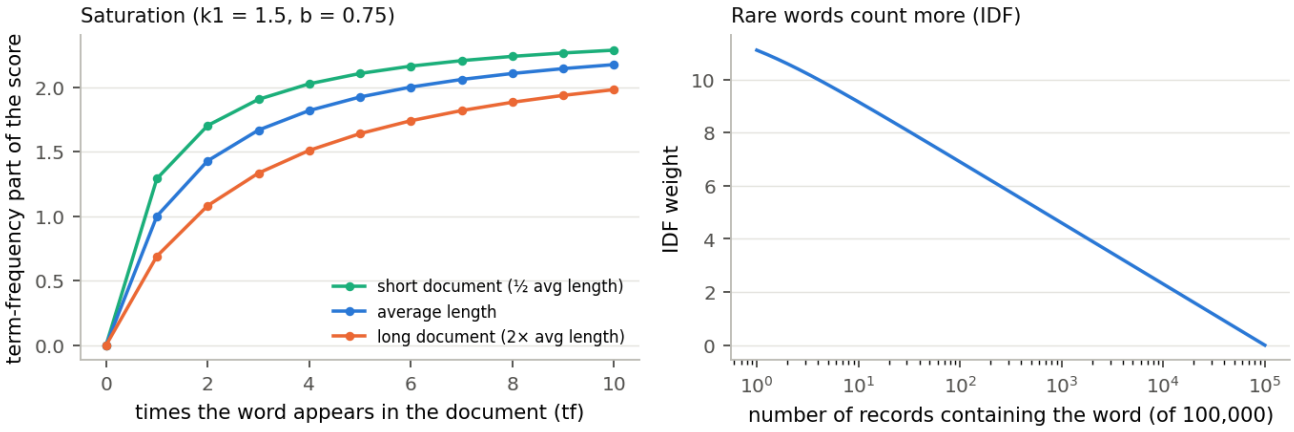


Figure 9.1 — Real parameters (k1 = 1.5, b = 0.75, 100,000 records): repetition saturates, long documents are damped, and rare words weigh more.

How your project uses it

- Implemented in `src/rag/bm25.py` (not a library), as a sparse matrix of 100,000 records × 139,815 terms, so a query is one fast column sum. Settings k1 = 1.5, b = 0.75 in `retrieval.yaml`.
- **Bigrams** (pairs of neighbouring words) are added: every record mentions 'latency' in its metric line, but only records with the condition contain the bigram `high_latency`, so phrase queries discriminate.
- BM25 shines on exact names (conditions, places, categories) — where embeddings are vague.

> **If the panel asks**
>
> "BM25 rewards rare query words and damps repetition and long documents. I added word pairs so 'high latency' matches records that actually have the condition, not every record that mentions latency."

10. Metric similarity and percentile rank

In one sentence. Metric similarity compares **numbers**: if the question says 'latency 150 ms' or 'high latency', records whose actual latency fits that statement score close to 1.

> **Everyday analogy**
> Shopping for a laptop under €800: a €790 laptop fits perfectly, an €850 one nearly fits, a €2,000 one does not — closeness on a number, not on words.

How it works

Stated value ("latency 150 ms"): similarity = 1 − |record value − 150| ÷ (P99 − P01), so it falls linearly as the record moves away from 150 ms across the metric's normal range.

Above / below ("latency above 120 ms"): 1 for every record beyond the value, decaying for records short of it.

Qualitative 'high': half the record's **percentile rank** (0 = best in the dataset, 1 = worst) plus half for actually crossing the condition threshold, so records that truly have the condition jump ahead.

'Normal': 1 at the median, falling towards both extremes. **Context** (weather, season, state mentioned in the text): 1 if equal, else 0.

The record's metric score is the average over everything the question mentioned.

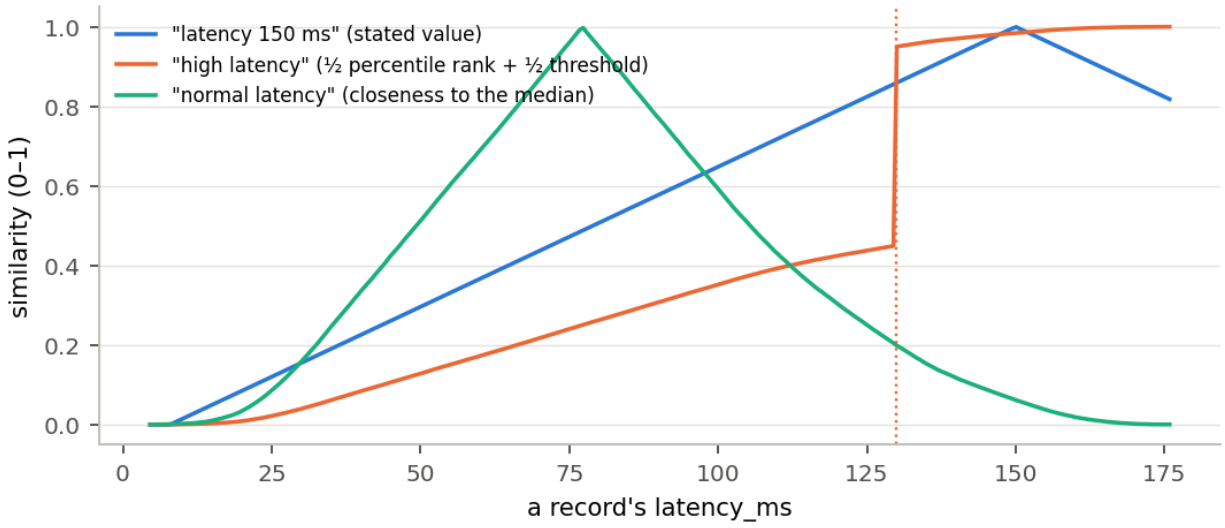


Figure 10.1 — Real: similarity of a record to three phrasings about latency, using your data's percentiles and the 129.92 ms threshold (dotted).

How your project uses it

`src/rag/metric_similarity.py`. Percentile ranks are computed against all 100,000 records, so 'high' always means high for this network.

The threshold half was added after testing: with rank alone, 'low throughput' preferred records just above the threshold, and precision@5 on one evaluation case was only 0.2. With the threshold indicator it reached 1.0. You should mention that this weight was tuned on your development cases.

> **If the panel asks**
> "Embeddings and keywords do not understand '150 ms', so a third score compares the record's actual values with what the user stated, using the dataset's own percentiles and thresholds."

11. Hybrid retrieval and query understanding

In one sentence. Hybrid retrieval combines the three signals — semantic, keyword and metric — into one ranking. **Query understanding** first reads the question with rules to find which metrics are described as bad, good or normal.

> **Everyday analogy**
> Three witnesses describe a suspect: one remembers the face (meaning), one the name tag (keywords), one the height (numbers). A detective weighs all three statements instead of trusting one.

How it works

1. Understand the query. `src/rag/query_parser.py` uses **regular expressions** (text patterns) to find phrases like 'declining handover success' (adverse), 'good signal strength' (favourable), 'normal user load' (normal), values ('above 12%') and context ('during snow'). Good/normal phrases are removed from the text sent to semantic and keyword search.

2. Apply filters (state, season, weather, category …) as a mask over all records.

3. Collect candidates: the top 300 from each method inside the filter.

4. Re-score every candidate exactly with all methods, and **min-max normalise** the semantic and keyword scores over the candidates (lowest → 0, highest → 1) so they are comparable.

5. Weighted sum. Weights 0.6 semantic, 0.4 keyword, 0.6 metric, re-normalised to the methods active for this query (e.g. 0.375 / 0.25 / 0.375). Subtract 0.15 for each condition the user called good that the record nevertheless has.

6. Final guarantee: every result is checked against the filters again before it is returned.

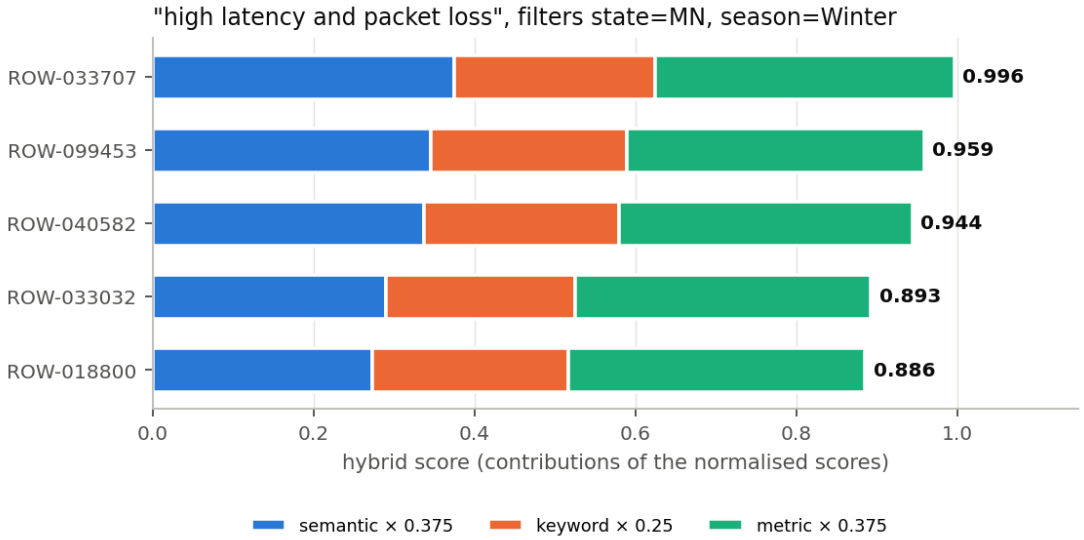


Figure 11.1 — Real: 'high latency and packet loss', filtered to Minnesota in winter (412 eligible records, 386 candidates). Each bar shows how the three signals add up.

How your project uses it

Top result of that search, `ROW-033707`: semantic 0.568, keyword 1.0 (raw BM25 4.83), metric 0.99 → hybrid 0.996.

A test checks seven filter combinations and proves that no returned record ever violates a filter.

Problem found

Real problem found by the LLM judge: 'normal user load' used to retrieve high-load records, because embeddings matched the topic 'user load'. The fix removes good/normal phrases from the search text and adds the 0.15 penalty.

If the panel asks

"Each method has a blind spot — meaning, exact words or numbers — so I combine all three with configurable weights after normalising them, and filters are enforced twice."

Part D — Machine learning

How the project trains models that estimate fault risk and throughput and cross-check the category — and how it proves those estimates are honest.

12. Supervised learning, train/test split and cross-validation

In one sentence. Supervised learning means learning from examples where the answer is known. A model is trained on most of the data and graded on a part it has never seen (the **test set**), because only performance on unseen data shows whether it really learned.

> **Everyday analogy**
> A student who memorises last year's exam answers scores 100% on those questions and fails the real exam. Grading on unseen questions is the only fair test.

How it works

Regression predicts a number (fault rate 44.08%). **Classification** predicts a category (Mobility / Handover).

Train/test split: shuffle the records, keep 80% for training and hide 20% for the final exam. A fixed **random seed** (random_state = 42) makes the shuffle identical every run, so results are reproducible.

Cross-validation (k-fold): split the training part into 5 folds; train 5 times, each time scoring on a different fold. If the 5 scores agree, the result is stable rather than lucky.

Overfitting is when a model fits the training data's noise and does worse on new data; the test set and cross-validation detect it.

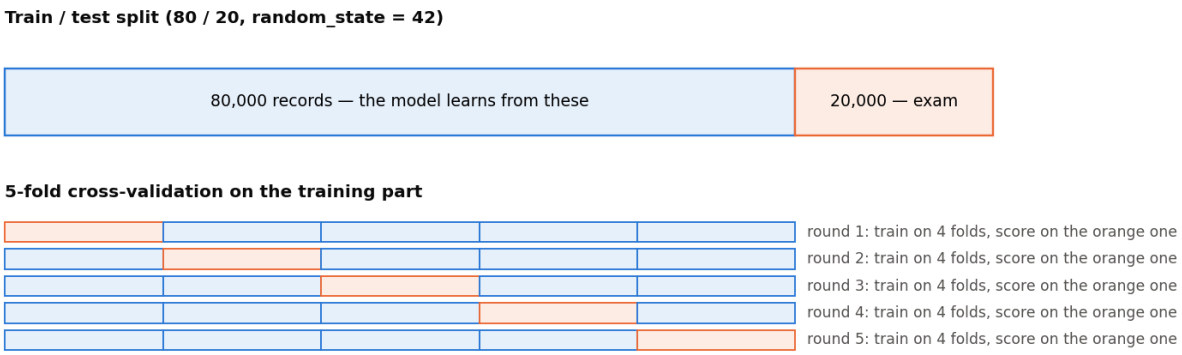


Figure 12.1 — The split and 5-fold cross-validation used in your project (illustration).

How your project uses it

Settings in `src/config/models.yaml`: test_size 0.2, random_state 42 → 80,000 training and 20,000 test records. Training code: `python -m src.services.models.train`.

The served fault-risk model's 5-fold cross-validation R^2 is 0.8848 ± 0.0006 — almost no variation between folds, so the test score is not luck.

The classifier's split is **stratified**: each category keeps the same share in training and test.

> **If the panel asks**
> "Every score I report is on 20,000 records the model never saw, and 5-fold cross-validation gives the same R^2 within ±0.0006."

13. Linear regression, logistic regression and feature encoding

In one sentence. Linear regression predicts a number as a weighted sum of the inputs; **logistic regression** does the same for categories by turning weighted sums into probabilities. Both are simple **baselines** that more powerful models must beat.

> **Everyday analogy**
> A taxi fare: a fixed start fee plus a price per kilometre plus a price per minute. Linear regression learns those prices from past trips.

How it works

Linear: prediction = $b_0 + b_1 \times$ latency $+ b_2 \times$ packet loss + ... The training finds the weights b that minimise the squared errors.

Logistic: one weighted sum per category, converted into probabilities that add up to 1 (softmax).

One-hot encoding turns a text category into numbers: weather = Rain becomes four columns weather_Clear 0, weather_Clouds 0, weather_Rain 1, weather_Snow 0.

Standard scaling (`StandardScaler`) rescales each feature to mean 0 and standard deviation 1, so a feature measured in thousands (users) does not dominate one measured in single digits (packet loss). Logistic regression needs it; tree models do not.

How your project uses it

- Both are trained as baselines in `src/services/models/fault_risk.py`, `performance.py` and `incident_classifier.py`.
- A real lesson: for fault risk, plain linear regression scores R^2 0.8848 — the same as the boosted trees — because the target is almost a linear formula of other columns (Chapter 17). Simple models reveal the structure of the data.
- For categories the rules are non-linear (thresholds, highest score wins), so logistic regression reaches only 65.1% agreement while tree models reach 97.4%.
- Unknown or missing weather becomes 'missing' in every one-hot column rather than a fake category.

> **If the panel asks**
> "I always train a linear baseline. For fault risk it matched the complex model, which told me the target is nearly a formula — that finding drove the leakage analysis."

14. Decision trees and random forests

In one sentence. A **decision tree** predicts by asking a sequence of yes/no questions about the inputs; a **random forest** averages hundreds of slightly different trees to get a steadier answer.

> **Everyday analogy**
> The game '20 questions': each question splits the possibilities in two until you are confident. A random forest is asking 200 friends to play and taking the average guess.

How it works

Training picks, at each step, the question (feature and cut-off) that best separates high from low targets. Each final box (leaf) predicts the average target of the training records that reach it.

A deep tree can memorise noise (overfitting). A random forest trains each tree on a random sample of rows and features and averages them, which cancels much of the noise.

Trees handle non-linear rules and interactions naturally and need no scaling.

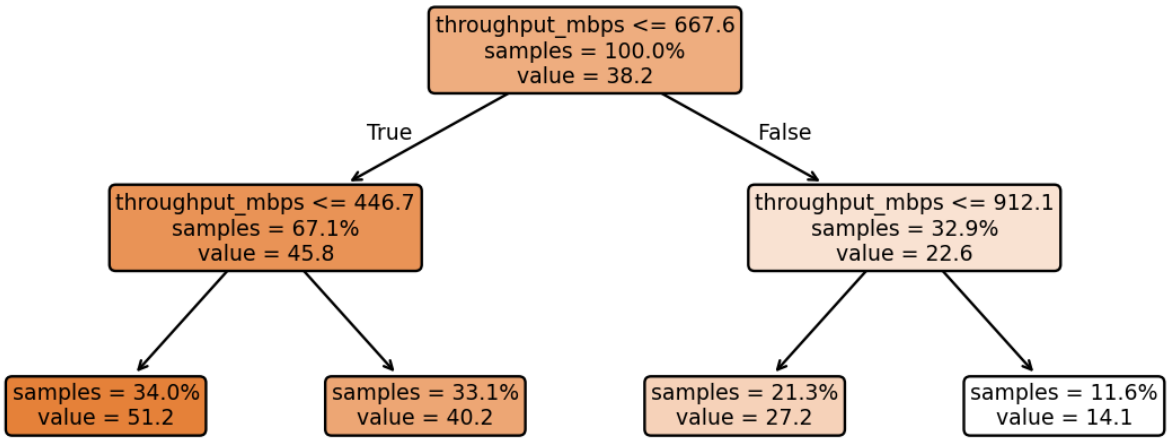


Figure 14.1 — Real: a 2-level tree fitted to your training data to predict fault_rate_pct. It discovers on its own that throughput separates high- from low-fault records (served models are far larger).

How your project uses it

Random forests are trained for comparison: fault risk R^2 0.8831, throughput R^2 0.3242, classifier 97.2% agreement (200 trees, `random_state = 42`).

They were not served because gradient boosting (next chapter) scored as well or slightly better and handles missing inputs natively.

> **If the panel asks**
> "Trees learn threshold-style rules from data; a forest averages many trees to reduce overfitting. I used forests as a comparison model."

15. Gradient boosting (HistGradientBoosting)

In one sentence. Gradient boosting builds trees one after another, each new small tree correcting the remaining errors of all previous ones. **HistGradientBoosting** is scikit-learn's fast version, and it is the model your project serves for all three tasks.

> **Everyday analogy**
> Learning darts with a coach: after every throw the coach says 'a bit higher and left'. Each small correction is weak, but hundreds of them add up to accuracy.

How it works

Start with a simple guess (the average). Compute each record's error. Fit a small tree that predicts those errors. Add a fraction of it (the **learning rate**, 0.05 in your fault model) to the prediction. Repeat (`max_iter` = 300 trees).

'**Hist**': numeric features are first grouped into at most 255 bins (a histogram), which makes finding the best split much faster on 80,000 rows.

Missing values are handled natively: the model learns which branch missing values should go down. This matters because engineers rarely type every KPI.

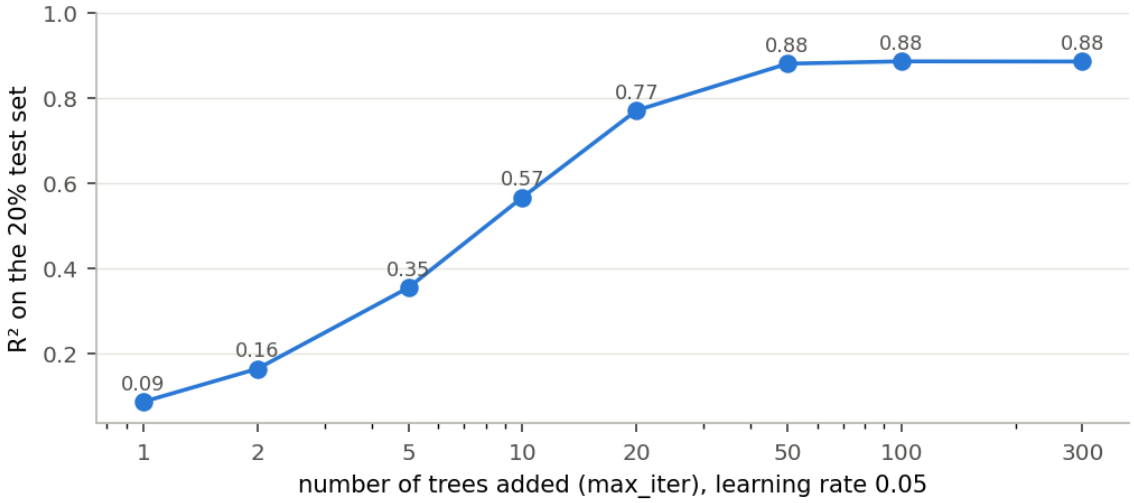


Figure 15.1 — Real: test-set R² of the fault-risk model as trees are added (trained on your 80,000 training records for this primer).

How your project uses it

The curve shows boosting at work: 1 tree explains R² 0.086, 10 trees 0.566, 50 trees 0.879, and from 100 trees on it stays at 0.885 — more trees no longer help, and the score does not drop either (no overfitting).

- Fault risk: `HistGradientBoostingRegressor(max_iter=300, learning_rate=0.05)`.
- Throughput: the same regressor; category: `HistGradientBoostingClassifier(max_iter=300, learning_rate=0.1)`.
- Models are saved with joblib in `data/processed/models/*.joblib` and reports in `docs/models/*.json`.

If the panel asks

"Gradient boosting adds small trees that each fix the previous errors. I serve the histogram version because it is accurate, fast and handles missing inputs, which partial user queries always have."

16. Measuring models: R², MAE, RMSE, accuracy, precision, recall, F1

In one sentence. Metrics turn 'is the model good?' into numbers. Regression uses error sizes and R²; classification uses the share of correct answers and how errors are distributed between categories.

Everyday analogy

Weather forecasts: MAE is 'on average I miss the temperature by 2 °C'; R² is 'how much better I am than always forecasting the seasonal average'.

How it works

MAE (mean absolute error): average size of the miss, in the target's unit (percentage points).

RMSE (root mean squared error): like MAE but punishes big misses more.

R² (coefficient of determination): 1 = perfect, 0 = no better than always predicting the average, below 0 = worse. It is the share of the target's variation the model explains.

Accuracy: share of correct categories. **Precision** for a category: of the records predicted as it, how many really are. **Recall**: of the records that really are it, how many were found. **F1**: the balance (harmonic mean) of the two. **Macro** averages treat every category equally, so small categories count as much as big ones.

A **confusion matrix** counts every (true, predicted) pair; the diagonal holds the correct answers.

How your project uses it

Model (served)	Metric on 20,000 unseen records	Meaning
Fault risk	R² 0.8849, MAE 3.7249 points, RMSE 4.7669	explains 88% of the variation; typical miss about 3.7 percentage points
Throughput	R² 0.3283, MAE 162 Mbps (baseline MAE 197)	moderate: weather drives throughput, so treat it as indicative
Category classifier	accuracy 97.4%, macro F1 0.976	agreement with the rule-derived categories, not real-world accuracy

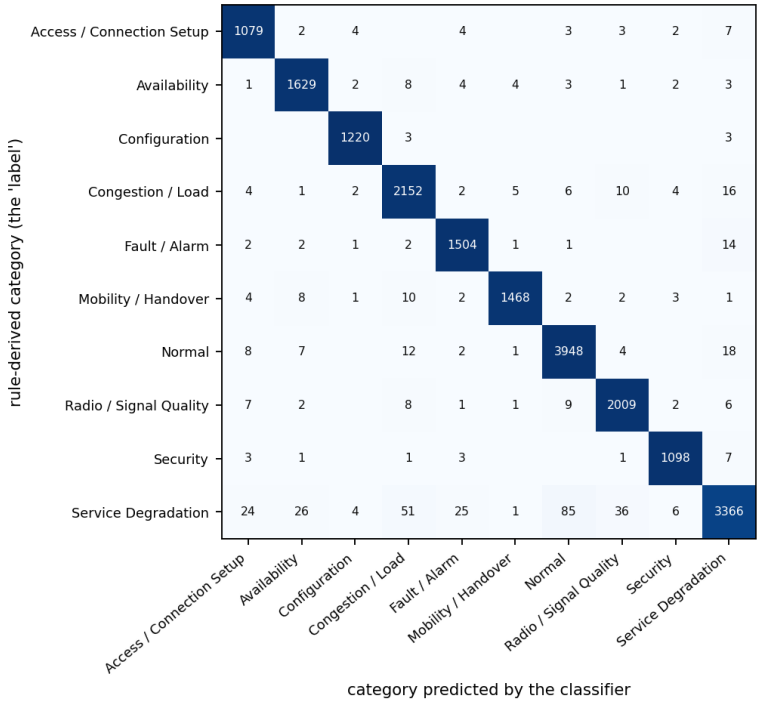


Figure 16.1 — Real: confusion matrix of the served classifier (rows = rule category, columns = prediction). Most errors involve Service Degradation, the broadest category.

Watch out

The classifier learns categories that your own rules produced, so 97% means 'the rules can be learned from raw metrics', not 'it diagnoses real incidents correctly'. Always say 'agreement'.

If the panel asks

"The fault model explains 88% of the variation with a typical error of 3.7 points. The classifier's 97% is agreement with my rules, which I report as a consistency check, not as accuracy."

17. Data leakage and feature sets

In one sentence. Data leakage is when a model is given information that contains the answer or would not exist at prediction time. It produces impressive scores that collapse in real use.

Everyday analogy

A student who has the answer key hidden under the exam paper: a perfect score, zero learning.

How it works

Leakage hides in two ways: features computed from the target (directly or as part of its formula), and features only known after the event (e.g. repair time after a fault).

The defence is to define allowed **feature sets** on purpose and to enforce them in code, not by memory.

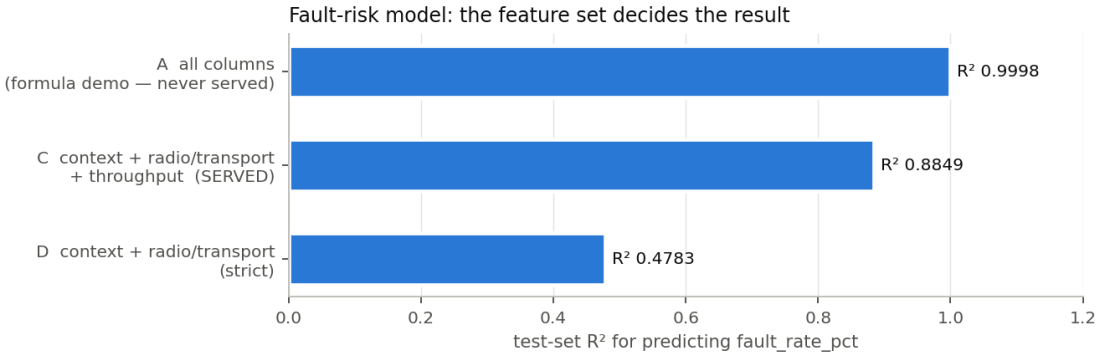


Figure 17.1 — Real: fault-risk R² for the three feature sets. The 0.9998 bar is the leakage demonstration and is never served.

How your project uses it

Feature set	What it contains	Test R² (linear / boosting)	Status
A_formula_demo	every column except the target	0.9998 / 0.9997	proof that the target is a formula — never served
C_primary	context + radio/transport KPIs + throughput	0.8848 / 0.8849	**served**
D_strict	C without throughput	0.48 / 0.4783	strictest reference

- Excluded from the served model (`models.yaml`): the seven columns that form the fault rate's formula (critical alarms, auth failures, security incidents, alarms, call drops, configuration success, parameter changes), MTTR (known only after a fault), every derived label and condition, locations and IDs.
- `check_leakage()` in `src/services/models/common.py` raises an error if any of these is used, and a test proves it.

> **If the panel asks**
> "With every column the model scores 0.9998 because it rediscovers the target's formula — that is leakage. The served model uses only information available before a fault and scores 0.885 honestly, and a code guard makes leakage impossible to reintroduce."

18. Prediction intervals and risk bands

In one sentence. A single predicted number hides uncertainty, so the project adds an **80% interval** around every fault-risk prediction and maps the prediction to a **risk level** (Low, Medium, High, Critical).

> **Everyday analogy**
> A weather app that says '18 °C, likely between 15 and 21 °C' is more useful than '18 °C' alone.

How it works

Residuals are the model's errors on the test set (actual − predicted). Their 10th and 90th percentiles are added to each new prediction, giving an interval that the actual value fell inside for 80% of test records.

Risk bands are cut-offs on the predicted rate, taken from the dataset's own distribution: below P25 is Low, below P75 Medium, below P90 High, otherwise Critical.

Input coverage is the share of the model's inputs the user actually provided. With few inputs the model can only return its baseline expectation, and the UI says so.

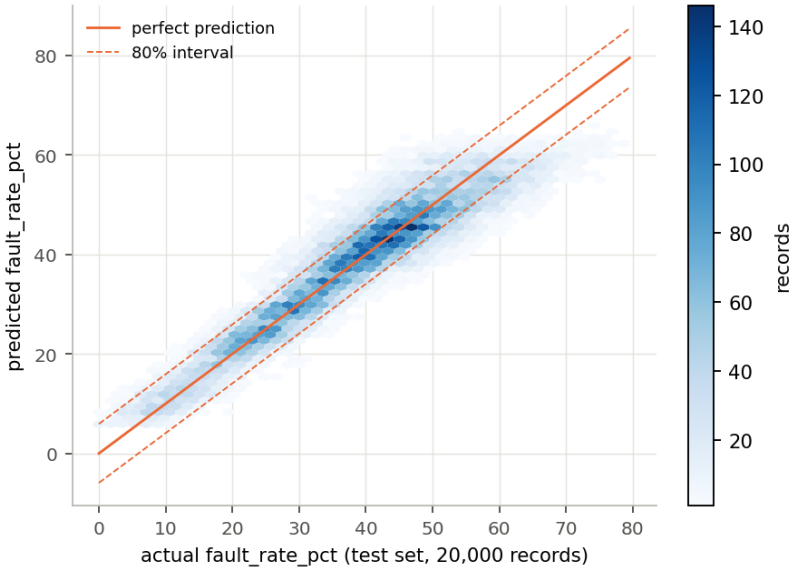


Figure 18.1 — Real: predicted vs actual fault rate for the 20,000 test records. The dashed lines are the 80% interval.

How your project uses it

Residual percentiles: -5.91 and +5.93 points. Risk bands: Low below 28.339, Medium below 48.152, High below 55.5134, else Critical.

Phase 30 example: 44.08% predicted, interval 38.17–50.01%, risk Medium; shown beside the 58.05% **observed** in the five similar records — two different things, never merged.

Watch out

The interval was calibrated on the same test residuals it is checked against, so its 80% coverage there is by construction. On truly new data it is an estimate, and it assumes errors are similar across the range.

If the panel asks

"Every prediction comes with an 80% range from the model's own test errors and a risk band from the data's percentiles, and it is always labelled as a prediction, separate from observed history."

Part E — LLM, RAG and agents

How the LLM is used safely, how answers are grounded in evidence, and how the analysis is organised into agents.

19. Large language models: gpt-oss-20b on NVIDIA

In one sentence. A **large language model (LLM)** is a neural network trained on enormous amounts of text to predict the next word; this makes it able to read instructions and write fluent text. Your project uses the open-weight model **gpt-oss-20b** (`openai/gpt-oss-20b`) through the NVIDIA API Catalog, an OpenAI-compatible endpoint. Claude (Anthropic) is supported as an alternative and was used during development.

> **Everyday analogy**
> An extremely well-read assistant who writes beautifully — and who will confidently fill gaps with plausible guesses unless you give them the documents and check their work.

How it works

Text is processed as **tokens** (pieces of words). Each request has a **system prompt** (standing rules) and a **user message** (the task and data).

Hallucination is fluent text that is not supported by the input — invented numbers, IDs or causes. It is the central risk of using LLMs for engineering decisions.

Structured outputs: instead of free text, the model must fill a fixed schema (fields and types). Your code passes a Pydantic schema to `client.messages.parse(…)` and receives a validated object, so there is no fragile text parsing.

How your project uses it

- **Provider abstraction** (`src/rag/llm_provider.py`): every LLM provider offers one method, `structured(system, user, schema)`. The NVIDIA provider calls gpt-oss-20b (the Anthropic provider can call Claude instead); the Offline provider returns nothing and the code falls back to templates. The whole system therefore works without an API key.
- Configuration only from `.env`: ANTHROPIC_API_KEY, LLM_MODEL, LLM_PROVIDER — never written in code.
- Errors are reported, not hidden: authentication, rate limits, bad requests (such as the 'credit balance too low' message) and network failures each return a clear status that the UI shows.
- The narrative schema has four fields: summary (≤ 3 sentences), diagnosis, one note per hypothesis id, and limitations.
- The system prompt's rules include: use only facts in the evidence; copy numbers exactly and compute nothing new; refer only to listed record IDs; treat causes as hypotheses; name no protocol details not in the evidence; never promise outcomes.

If the panel asks

"The LLM never decides facts. It receives an evidence package and fills a fixed schema with an explanation; if it is unavailable the system uses a deterministic template and still works."

20. Retrieval-augmented generation (RAG)

In one sentence. RAG means retrieving relevant documents first and giving them to the LLM, so its answer is based on that evidence rather than on its memory.

Everyday analogy

An open-book exam: the student must answer from the provided pages, and the grader can check every sentence against them.

How it works

Retrieve → assemble an evidence package → generate → check. Without retrieval an LLM can only answer from general knowledge; with retrieval it can discuss your specific records.

Most RAG systems stop at 'generate'. Your project adds a check step and goes further: the classification, risk, hypotheses and actions are computed by code **before** the LLM is called, so the LLM explains conclusions instead of making them.

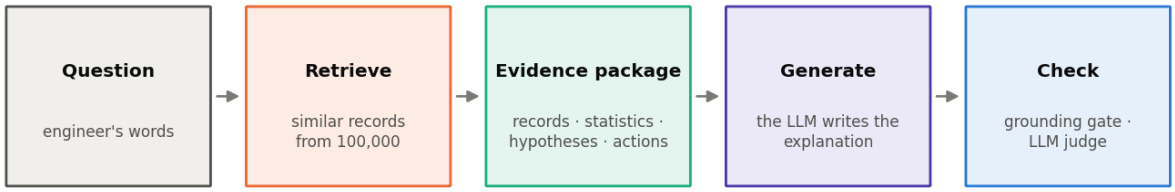


Figure 20.1 — Retrieval-augmented generation as used in your project (illustration).

How your project uses it

The evidence package (`evidence_package()` in `src/rag/narrative.py`) contains: the query; values and conditions the operator stated; the retrieved records; statistics of those records; the dataset baseline; classification and priority; predicted and historical fault risk; the ranked hypotheses; and the recommended actions.

The very same package is used to check the LLM's answer, so 'grounded' has a precise meaning: supported by this package.

If the panel asks

"It is RAG with deterministic analysis in the middle: retrieval finds evidence, code draws the conclusions, and the LLM only explains them from that evidence."

21. The grounding gate: catching hallucinations

In one sentence. A deterministic check in code reads every sentence the LLM writes and rejects it if it contains a number, record ID, protocol term or promise that the evidence does not support.

> **Everyday analogy**
> A fact-checker at a newspaper who has the source documents and strikes out every figure that cannot be found in them — before publication.

How it works

Numbers: each number in the text must match an evidence value *at the precision written* — '74.3' matches 74.336, but '99.9' does not match 99.5. Shares like 0.65 may be quoted as 65%. Small counting words ('two hypotheses') are allowed, but not with a unit ('4–5 points').

Record IDs must be among the retrieved records.

Unsupported specifics: terms such as X2, Xn, RACH, AMF, MME or A3 are rejected unless they appear in the evidence — the dataset has no such counters.

Guarantees: 'will increase to 95%', 'within 24 hours', 'guaranteed', 'confirmed root cause' are rejected.

If a check fails, the LLM gets **one retry** with the violations listed; if it fails again, the deterministic template replaces it.

How your project uses it

Code: `src/rag/grounding.py` (`check_text`). Real test with a deliberately hallucinated paragraph:

```
Handover success will increase to 97% after the fix, which should take about 6 hours. This matches
incident ROW-123456, where 12,500 users were affected. The likely cause is an X2 link failure
combined with RACH congestion. Fault rate is 4.2 points above baseline.
```

Caught: the invented record ID, the 12,500 users, X2 and RACH, and the promised increase. Not caught: '97%', '6 hours' and '4.2 points', because those numbers happen to equal unrelated evidence values. The check is value-based — that is why an LLM judge reviews meaning as a second layer.

> **If the panel asks**
> "Every number, ID and protocol term the LLM writes is checked against the evidence by code; failures get one retry and then a template. I also know its limit: a number that coincidentally matches unrelated evidence can pass."

22. LLM-as-a-judge

In one sentence. A second LLM call reviews the finished analysis against the evidence and answers fixed questions — is the diagnosis supported, is it grounded, are the actions relevant, is anything hallucinated.

> **Everyday analogy**
> A second engineer reviewing a colleague's report before it is sent, with the same case file in hand.

How it works

Code checks are precise but literal; a judge understands meaning (e.g. 'this sentence draws a conclusion the evidence does not support').

The judge also returns structured output: true/false verdicts, completeness, a score and a list of issues.

The final verdict combines both: a check passes only if the deterministic check **and** the judge agree; hallucination is flagged if either finds it.

How your project uses it

- Code: `src/rag/judge.py`. Deterministic checks always run; the judge runs when the LLM is enabled.
- The UI's 'Automatic review' shows four checks: evidence grounded, diagnosis supported, recommendations relevant, no hallucination detected — plus the method ('deterministic only' or 'deterministic + LLM judge').
- A real finding by the judge: 'normal user load' retrieved high-load records (Chapter 11).

> **Watch out**
> On the Phase 30 demo query, 'Recommendations relevant' fails. The two weather actions come from a lower-ranked hypothesis supported by the similar records, not by a condition in the question, and the check is deliberately strict about that.

> **If the panel asks**
> "Two layers review every answer: exact code checks and an LLM judge for meaning. A check only passes if both agree."

23. Root-cause hypotheses: the playbook heuristic

In one sentence. Possible causes are listed in a **playbook** written as configuration; each is scored with a documented formula and reported as an engineering **hypothesis**, never a confirmed cause.

> **Everyday analogy**
> A doctor's differential diagnosis: 'most likely X, consider Y and Z', each with the tests that would confirm or rule it out.

How it works

A hypothesis is considered when one of its trigger conditions is present.

Confidence = 0.20 + 0.45 × (share of its evidence weight that is present) + 0.25 × (share of similar records showing its key evidence) − 0.25 per contradicting item, clipped to 0.05–0.95.

Each hypothesis carries the checks to run, the actions to consider, and the additional evidence (not in this dataset) that would confirm or refute it.

This is a **heuristic**: a transparent rule of thumb, not a calibrated probability.

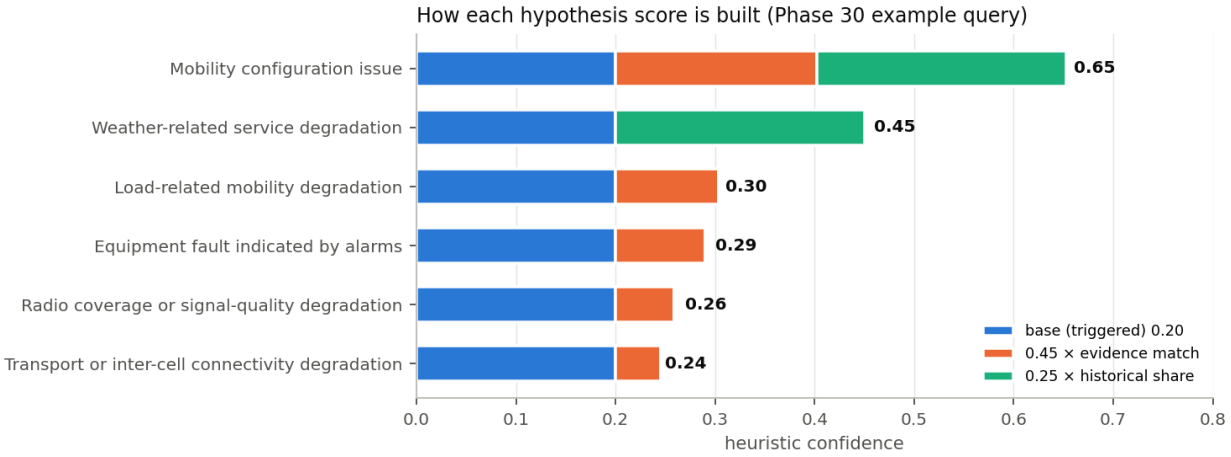


Figure 23.1 — Real: how each hypothesis score was built for the Phase 30 query.

How your project uses it

Phase 30 example: mobility configuration issue = 0.20 + 0.45 × 0.45 + 0.25 × 1.0 = 0.65 — low handover success was stated, and all 5 similar records show it.

Recommendations are built from the hypotheses, each linked to the conditions it is based on, plus monitoring actions compared with the dataset baseline. Disclaimer always attached: options to evaluate, no KPI improvement predicted.

Config: `src/config/playbook.yaml`; code: `src/services/root_cause.py`, `recommendations.py`.

> **If the panel asks**
> "0.65 is a documented heuristic: base 0.20, plus evidence match, plus support from similar records, minus contradictions. It ranks hypotheses; it is not a probability."

24. Agents and orchestration

In one sentence. An **agent** here is a named step with one responsibility that reads a shared state, adds its result and records what it did. An **orchestrator** runs the six agents in a fixed order.

> **Everyday analogy**
> An assembly line with stations: each station does one job on the same product and stamps a logbook.

How it works

In the AI world 'agent' often means an LLM that decides its own next steps in a loop. Your project deliberately does **not** do that: the order is fixed, nothing loops, and most agents use no LLM. That makes every run predictable and testable.

Shared state: a typed object (`AnalysisState`) holding the request, the growing result and the trace. Each agent declares which inputs it requires and fails loudly if they are missing.

Trace: each agent records its status, duration and a one-line summary — shown in the UI.



Agent order and measured duration for the Phase 30 query (offline mode). With the LLM enabled the Recommendation and Validation agents add LLM calls (~10-40 s).

Figure 24.1 — Real: the six agents and their timings for the Phase 30 query (offline mode; the LLM adds about 10–40 s).

How your project uses it

Agent	Job
Retrieval	query understanding, filters, hybrid search, evidence extraction
Classification	conditions → category, severity (and the ML cross-check)
Risk	fault-risk prediction, priority
Diagnosis	playbook hypotheses with confidence
Recommendation	actions, checks, monitoring; LLM narrative + grounding gate
Validation	deterministic checks + LLM judge, provenance, text report

Guardrails run before the first agent. A test proves the agents give exactly the same result as the single-pipeline version in `src/rag/pipeline.py`. LangGraph was considered; for six linear steps a plain orchestrator is simpler to explain and test.

> **If the panel asks**
> "Six fixed agents on a typed shared state, each timed and traced. No autonomous loops — and a test proves they produce exactly the single pipeline's result."

25. Guardrails and prompt injection

In one sentence. Guardrails are checks that run on the user's input before anything else; they reject attempts to misuse the system, such as **prompt injection** — text that tries to override the assistant's instructions.

> **Everyday analogy**
> Security at a building entrance: before you reach any office, the guard checks you are here for a legitimate reason and not carrying anything dangerous.

How it works

Your guardrails are **regular expressions** — precise text patterns — so every decision is explainable and testable, and no LLM is involved.

Input is first **sanitised**: Unicode normalised, invisible control characters removed, whitespace collapsed (attackers hide instructions in invisible characters).

How your project uses it

Category	Example that is rejected
empty / too_long	nothing to analyse, or more than 1,000 characters
prompt_injection	'Ignore your instructions…', 'you are now…', 'act as…', fake <system> tags
secret_request	API keys, passwords, tokens, .env, 'reveal your prompt'
unsupported	delete data, restart a cell or site, run commands, push to the live network
off_topic	no telecom-network vocabulary at all

Code: `src/services/guardrails.py`. The API answers a rejected query with HTTP 422 and a code such as `guardrail_prompt_injection`; the web UI shows the reason and how to rephrase.

> **If the panel asks**
> "Inputs are sanitised and checked with explainable patterns before any retrieval or LLM call; it is a decision-support tool, so requests to act on the network are refused."

26. Provenance: where every statement comes from

In one sentence. Provenance means recording the source of each statement in an answer, so a reader can tell a measured fact from a prediction or a hypothesis.

> **Everyday analogy**
> Footnotes in a research paper: every claim points to where it came from.

How it works

Each statement in the final analysis is stored with one of seven source types and the evidence behind it (condition, record IDs, values).

How your project uses it

Source type	Meaning	Example
USER_INPUT	stated by the operator	low handover success reported in the query
RAW_DATA	a measured dataset value	ROW-098813: latency 120.16 ms
DERIVED_FEATURE	computed by documented rules	category Mobility / Handover, severity MEDIUM
HISTORICAL_EVIDENCE	a pattern across similar records	low handover in 5 of 5 similar records
ML_PREDICTION	a model estimate	predicted fault risk Medium (44.08%)
ENGINEERING_HYPOTHESIS	a possible cause	mobility configuration issue (0.65)
RECOMMENDATION	an action to consider	audit neighbour relationships

The web UI's Sources section lists every statement with its source and lets you filter by type.

> **If the panel asks**
> "Every statement is tagged with one of seven source types, so a measured value can never be confused with a prediction or a hypothesis."

Part F — Quality: testing and evaluation

How the project proves that it works and that it does not invent things.

27. Automated testing with pytest

In one sentence. An **automated test** is a small program that runs part of the system and checks the result; **pytest** is the standard Python tool that finds and runs them all with one command.

> **Everyday analogy**
> A pre-flight checklist: every item is checked the same way before every flight, so nothing depends on someone remembering.

How it works

Unit tests check one function (e.g. the BM25 formula). **Integration tests** run several parts together (e.g. a full API request). **Regression tests** pin a fixed bug so it can never return.

A test **asserts** something that must be true; if it is false, the test fails and names the problem.

Parametrised tests run the same check on many inputs, which is why 130 test functions produce 222 test runs.

How your project uses it

Run with `python -m pytest`: 222 tests, one file per phase group in `tests/`, about 2–3 minutes. All pass.

The five proofs the specification demanded	How a test proves it
No raw value fabricated	every processed value is compared with the raw CSV text
Filters never violated	seven filter combinations; every returned record is checked
Retrieved IDs are real records	every ID returned by search exists in the dataset
LLM cannot add unsupported numbers	a deliberately lying fake LLM is rejected by the grounding gate
No target leakage	the leakage guard rejects forbidden features; the served model is checked

The React UI has its own 6 unit tests (Vitest) for formatting, filters and input validation: `npm test` in `web/`.

> **If the panel asks**
> "222 automated tests, including the five proofs the specification asked for — for example, every processed value is compared with the original file."

28. Evaluating retrieval and answers

In one sentence. An **evaluation set** is a fixed list of questions with the expected outcomes; running the system on it and scoring the results shows how well the whole pipeline works, not just single functions.

> **Everyday analogy**
> A driving test route: the same streets and situations for every candidate, scored the same way.

How it works

Precision@k: of the top k results, how many are relevant. Your retrieval precision counts a record as relevant if it shows *all* the expected conditions.

Average precision: rewards putting relevant results at the top, not just somewhere in the list (also called contextual precision).

Recall: of everything that should be found, how much was found. **Condition recall**: did the query parser recognise every expected condition?

Development set: a set used while building. Tuning on it makes scores optimistic; a separate held-out set would be needed for an unbiased number.

How your project uses it

8 cases in `tests/evaluation/eval_cases.json`, one per scenario in the specification (e.g. 'high latency and high packet loss' → expected category Service Degradation, expected hypothesis transport connectivity). Run: `python -m tests.evaluation.run_eval`.

Deterministic metric	Result	Meaning
condition_recall	1.00	every expected condition recognised
retrieval_precision	1.00	every retrieved record shows all expected conditions
average_precision	1.00	relevant records ranked first
category_match	1.00	derived category as expected
hypothesis_hit	1.00	an expected hypothesis in the top 2
grounded	0.75	explanation passed the grounding gate, no hallucination

> **Watch out**
> These are development scores: one retrieval weight was tuned on these same 8 cases. Say so — it shows you understand evaluation.

> **If the panel asks**
> "Eight specification scenarios score 1.00 on every deterministic metric. They are development scores, because I tuned one weight on them; a held-out set grown from feedback is future work."

29. DeepEval: LLM-judged quality metrics

In one sentence. DeepEval is an open-source library of evaluation metrics for LLM applications; an LLM acts as an independent judge and scores each answer against its question and retrieved context.

> **Everyday analogy**
> An external examiner who reads both the student's essay and the sources it was meant to use.

How it works

Faithfulness: share of the answer's claims that are supported by the retrieved context.

Answer relevancy: share of the answer's statements that address the question.

Contextual precision: are the relevant retrieved items ranked above the irrelevant ones?

Contextual relevancy: share of the retrieved context that is relevant to the question.

How your project uses it

Run with `python -m tests.evaluation.run_eval --deepeval` (uses API calls), with the configured LLM as the judge.

DeepEval metric	Score	Reading
Faithfulness	0.93	share of the answer's claims supported by the retrieved evidence
Answer relevancy	0.89	answers stay on the question
Contextual precision	1.00	relevant records come first
Contextual relevancy	0.28	lower by design: each record's text lists every metric, much of it unrelated to a given question

> **If the panel asks**
> "With the LLM as an independent judge, faithfulness is 0.93 and contextual precision 1.00. Contextual relevancy is lower (0.28) because each record description includes all its metrics, not only the relevant ones."

Part G — Serving and interface

How the analysis reaches the engineer: an API, a web interface, and the supporting tools.

30. APIs: REST, FastAPI and Pydantic

In one sentence. An **API** (application programming interface) lets one program ask another for something over the network. Your backend is a **REST** API built with **FastAPI**; **Pydantic** checks that every request and response has the right shape.

> **Everyday analogy**
> A restaurant: the menu lists what you can order (endpoints), the waiter takes a written order in a fixed format (JSON request) and brings back a dish (JSON response) — or tells you clearly what cannot be served (an error code).

How it works

HTTP is the protocol of the web. A request has a method (GET to read, POST to send data), a path (`/api/v1/analyze`) and often a **JSON** body — structured text of keys and values.

Status codes: 200 OK, 201 created, 404 not found, 422 the request was understood but is invalid, 500 server error.

FastAPI turns Python functions into endpoints and generates interactive documentation at `/docs` automatically.

Pydantic models declare fields, types and limits (e.g. packet loss between 0 and 100); invalid requests are rejected with a precise message before any analysis runs.

How your project uses it

Endpoint	Purpose
POST /api/v1/analyze	the full multi-agent analysis (main endpoint)
POST /api/v1/search	hybrid retrieval with filters
POST /api/v1/classify · /risk · /recommend	parts of the analysis on their own
GET /api/v1/records/{id}	one full record, raw and derived, labelled
GET /api/v1/reference	labels, units, ranges and thresholds for the web UI
POST /api/v1/feedback	store a useful / not-useful rating
GET /health	status of data, indexes, models and the LLM provider

Code: `src/main.py` and `src/api/schemas.py`. Every error has the same structure (code, message, details, request ID), and every response carries an X-Request-ID header for tracing.

> **If the panel asks**
> "FastAPI exposes the analysis as REST endpoints; Pydantic validates every request, so a value like packet loss 150% is rejected with a clear 422 error before any work is done."

31. The web interface: React, TypeScript and Vite (and Streamlit)

In one sentence. React is a JavaScript library for building interactive web pages from reusable components; **TypeScript** adds types to JavaScript so mistakes are caught before the page runs; **Vite** builds the code into small files a browser can load.

> **Everyday analogy**
> LEGO: React components are bricks (a badge, a table, a form field) that snap together into pages; TypeScript is the shape of each brick that prevents wrong pieces from fitting.

How it works

A **component** is a function that returns what to show for given data. When the data changes (**state**), React updates only the affected parts of the page.

A **single-page application (SPA)** loads once and then switches pages in the browser; the **router** maps addresses like `/ui/record/ROW-000001` to pages.

The browser calls the API with `fetch` and shows the JSON it gets back. The UI never computes analysis results itself.

Streamlit is a Python library that turns a script into a web page with very little code; it is ideal for prototypes and is the interface your specification asked for.

How your project uses it

- React app in `web/` with four pages: Analyze a case, Search records, Record details, Assess measurements. FastAPI serves the built files at `/ui`.
- Response types are declared in `web/src/lib/types.ts`, so the code cannot read a field the API does not return.
- Labels, units and thresholds come from `GET /api/v1/reference`, which reads the same YAML files as the backend — the browser never restates a number.
- Design: one accent colour, status always shown as text plus colour, light and dark themes, keyboard focus, and layouts that work at phone width.
- The Streamlit app (`frontend.py`) remains as the specified Phase 28 interface.

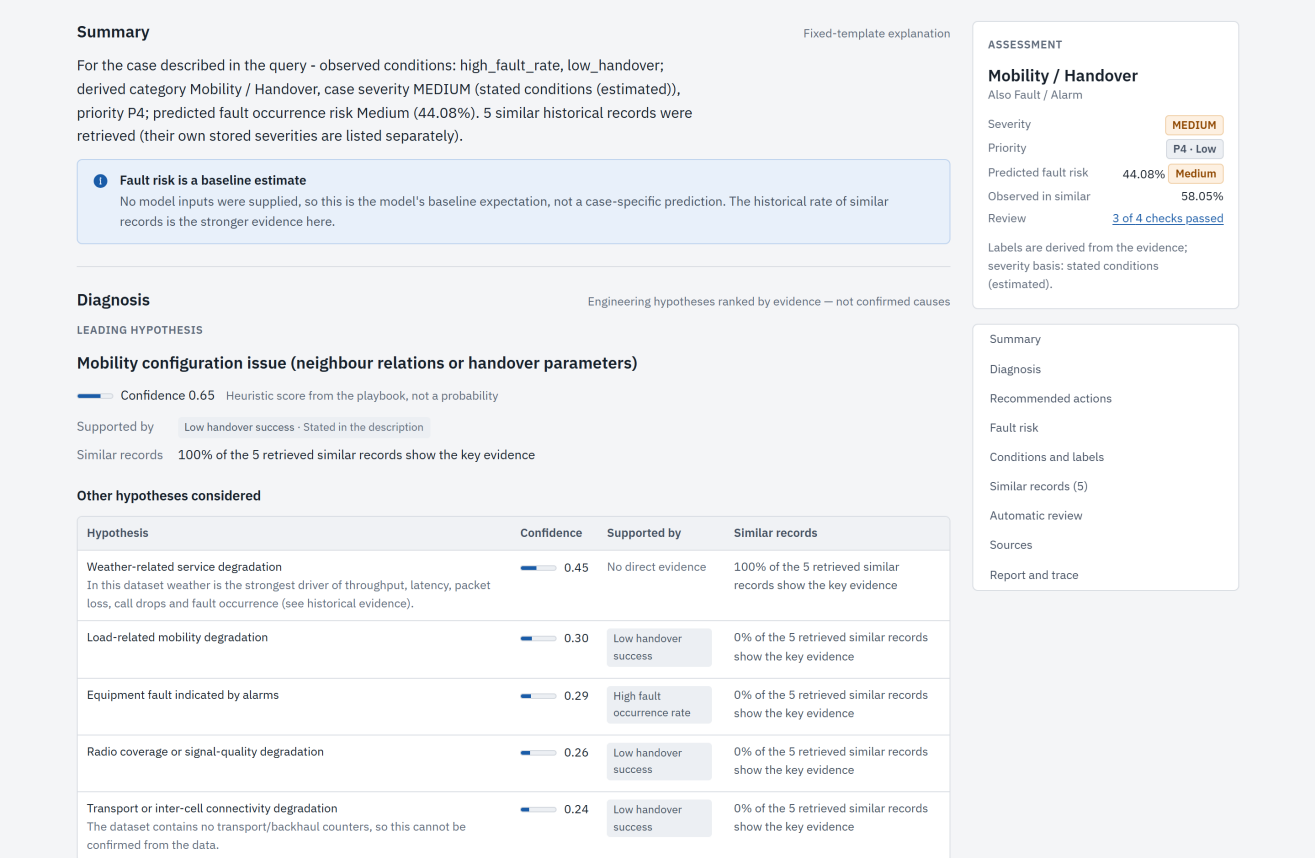


Figure 31.1 — The React interface for the Phase 30 query: the answer first, with the assessment panel pinned on the right.

> **If the panel asks**
> "The React interface is a client of the same API: it shows the answer first, keeps measured, derived and predicted values visibly apart, and takes every threshold from the backend."

32. Supporting tools: YAML, .env, SQLite and Docker

In one sentence. Four small tools keep the project configurable, safe, reproducible and easy to run.

> **Everyday analogy**
> The settings menu (YAML), a locked drawer for keys (.env), a notebook for comments (SQLite), and a shipping container that holds everything needed to run (Docker).

How it works

YAML is a human-readable configuration format. Keeping thresholds, weights and rules in YAML means behaviour can change without touching code.

.env holds secrets such as the API key outside the code; it is never committed or shared.

SQLite is a complete database stored in a single file — no server needed.

Docker packages the application and its exact dependencies into an image that runs the same on any machine; **docker-compose** starts several containers (API, UI) together.

How your project uses it

- YAML: `src/config/` — schema, thresholds, taxonomy, severity, priority, retrieval, models, playbook.
- .env: `ANTHROPIC_API_KEY`, `LLM_MODEL`, `LLM_PROVIDER`; a template without secrets is in `.env.example`.
- SQLite: `data/feedback/feedback.db` stores ratings with the query, retrieved IDs, diagnosis and comment. Nothing is retrained automatically; `python -m src.services.feedback export` exports cases for human review before they join the evaluation set.
- Docker: one image builds the React UI and installs the Python stack; `docker compose up` starts the API (with the UI at /ui) and the Streamlit app.

> **If the panel asks**
> "Every rule is in YAML, secrets are only in .env, feedback goes to SQLite for review rather than automatic retraining, and Docker makes the setup reproducible."

Cheat sheet — one sentence per technology

Read this the morning of the presentation.

Technology	Say it in one sentence
Rules vs ML vs LLM	Rules decide facts, ML estimates with measured error, the LLM only explains.
Percentile conditions	A condition means the worst 10% of this network's own history.
Category / severity / priority	Documented point systems in YAML: what kind, how bad, how urgent.
Embeddings (MiniLM)	Text turned into 384 numbers so similar meanings are close.
ChromaDB + HNSW	A vector database that finds close vectors fast, with metadata filters.
BM25	Keyword ranking that rewards rare words and damps repetition.
Metric similarity	Compares the record's actual numbers with what the user stated.
Hybrid retrieval	Normalised, weighted mix of meaning, keywords and numbers; filters enforced twice.
Train/test split, CV	Scores come from 20,000 unseen records; 5-fold CV confirms stability.
Gradient boosting	Many small trees, each correcting the previous errors; handles missing inputs.
R^2, MAE	R^2 = share of variation explained (0.885); MAE = typical miss (3.7 points).
Leakage	0.9998 rediscovers the target's formula; the served 0.885 uses only pre-fault data.
Prediction interval	An 80% range from the model's own test errors.
LLM + structured output	The LLM fills a fixed schema from the evidence package only.
RAG	Retrieve evidence, compute conclusions in code, let the LLM explain them.
Grounding gate	Code rejects any number, ID, protocol term or promise not in the evidence.
LLM-as-a-judge	A second review of meaning; a check passes only if code and judge agree.
Playbook confidence	0.20 + evidence match + similar-record support − contradictions; a heuristic.
Agents	Six fixed steps on a shared state, traced; no autonomous loops.
Guardrails	Explainable patterns reject injection, secret requests and off-topic input.
Provenance	Seven source types so facts, predictions and hypotheses never blur.
Testing / evaluation	222 tests incl. five proofs; 8 dev cases at 1.00 on retrieval; DeepEval with an LLM judge.
FastAPI + Pydantic	REST endpoints with validated inputs and structured errors.
React + TypeScript	A typed web client that shows the answer first and restates no numbers.

Glossary

Term	Meaning
API	A defined way for one program to request something from another over the network.
Average precision	A ranking score that rewards placing relevant results at the top.
Baseline	A simple reference result a better method must beat (e.g. always predicting the average).
Bigram	Two neighbouring words treated as one token, e.g. high_latency.
BM25	A keyword-ranking formula based on term frequency, rarity and document length.
Classification	Predicting a category.
Condition	A named true/false flag derived from one metric and a documented threshold.
Confusion matrix	A table counting every (true category, predicted category) pair.
Cosine similarity	How closely two vectors point the same way; 1 = same direction.
Cross-validation	Training and scoring several times on different splits to check stability.
Data leakage	Giving a model information that contains the answer or arrives after the event.
Decision tree	A model that predicts through a sequence of yes/no questions.
Derived	Computed from raw data by your own rules — an interpretation, not a measurement.
Deterministic	Always gives the same output for the same input.
Embedding	A list of numbers representing the meaning of a text.
Endpoint	One address of an API, e.g. POST /api/v1/analyze.
F1 score	The balance of precision and recall.
Feature	An input column used by a model.
Filter (metadata)	A restriction such as state = MN applied before ranking.
Gradient boosting	Building trees one after another, each correcting the remaining errors.
Grounding	Checking that generated text is supported by the given evidence.
Guardrail	A check on input or output that blocks misuse.
Hallucination	Fluent model output that is not supported by its input.
Heuristic	A transparent rule of thumb, not a calibrated probability.
HNSW	A layered graph index for fast approximate nearest-neighbour search.
Hybrid retrieval	Combining several search signals into one ranking.
Hypothesis	A possible cause that the evidence supports but does not confirm.
IDF	Inverse document frequency: rare words get higher weight.
Inference	Using a trained model on a new case.
JSON	A text format of keys and values used by web APIs.
Label	The known answer attached to a training example.
LLM	Large language model: a neural network trained on text to read and write language.
MAE / RMSE	Average size of prediction errors; RMSE punishes large errors more.
Min-max normalisation	Rescaling scores so the lowest is 0 and the highest is 1.
One-hot encoding	Turning a category into several 0/1 columns.
Overfitting	Fitting noise in the training data and doing worse on new data.
Parquet	A compact, typed, column-based file format for tables.

Term	Meaning
Percentile	The value below which a given share of the data falls (P90: 90%).
Precision / recall	Of predicted positives, how many are right / of real positives, how many were found.
Prompt injection	Input that tries to override an LLM system's instructions.
Provenance	The recorded source of a statement.
Pydantic	A Python library that validates data against declared types and limits.
R^2	Share of the target's variation a regression model explains (1 = perfect).
RAG	Retrieval-augmented generation: retrieve evidence, then generate from it.
Random forest	An average of many decision trees trained on random samples.
Regex	Regular expression: a precise pattern for matching text.
Regression	Predicting a number.
Residual	Actual value minus predicted value.
REST	A common style of web API using HTTP methods on resource paths.
Schema	The written contract for data: fields, types, units and valid ranges.
Structured output	LLM output forced into a fixed schema instead of free text.
Supervised learning	Learning from examples whose answers are known.
System prompt	Standing instructions given to an LLM for every request.
Target	The column a model learns to predict.
Test set	Data hidden from training and used only to score the model.
Token	A piece of a word; the unit LLMs read and write.
Transformer	The neural-network architecture behind modern language models and embeddings.
Vector database	A store that finds the vectors closest to a query vector.
YAML	A human-readable configuration file format.